



GROUP 2

End-to-End Traffic Data Pipeline for Ho Chi Minh City: Acquisition, Persistence and Analytical Insights

Data Report

November 2025

Contents

1	Introduction	1
1.1	Problem Definition and Proposed Solution	1
1.2	Overview of System Modules	1
1.2.1	Routing Module	2
1.2.2	Bus Map Module	2
1.2.3	SOS Map Module	2
1.2.4	Traffic Congestion Detection Module	2
1.3	Motivation for Unified Data Processing Pipeline	3
2	Data Collection Methods and Sources	4
2.1	Data Sources	4
2.2	Data Collection Methods	4
2.2.1	Bus Map Data	4
2.2.2	Traffic Camera Data	5
3	Database Management	8
3.1	Database Architecture	8
3.1.1	Relational Database (MySQL)	8
3.1.2	Object Storage (Google Drive Storage)	8
3.2	Schema Design	8
3.2.1	Users Table	9
3.2.2	User Locations Table	9
3.2.3	User Routes Table	10
3.2.4	SOS Alerts Table	10
3.3	Analytics Database	11
3.3.1	Primary Data Storage Schema	11
3.3.2	Analytics Metrics and Aggregated Data	13
3.3.3	Cache Layer for Performance Optimization	14
3.3.4	Data Export and Reporting Formats	15
4	Data Cleaning and Preprocessing Workflow	17
4.1	Bus Map Data Cleaning and Preprocessing	17
4.2	Traffic Camera Data Cleaning and Preprocessing	17
5	Exploratory Data Analysis	19
5.1	Descriptive Statistics	19
5.1.1	Geospatial Distribution	19
5.1.2	Numerical Variables	21
5.2	Analysis of Relationships	23
6	EDA For Unstructured Data	26
6.1	Traffic Camera Image Analysis	26

6.2	Traffic Congestion Data Pipeline	27
-----	--	----

Chapter 1

Introduction

1.1 Problem Definition and Proposed Solution

Travelers, from leisure visitors to business professionals, face significant and costly friction in planning and executing urban itineraries, which stems from a core problem of lacking nuances and fragmented data. Critical decision-making data on bus transit schedules, live traffic, and operating hours exists in disconnected silos. The system integrates multiple data sources to support intelligent transportation services, emergency response, and traffic management. This forces users to manually compare multiple apps, leading to:

- Inefficient planning under complex constraints (time, budget, transfers).
- Suboptimal in-the-moment transport choices, failing to balance cost, ETA.
- Reactive, unactionable responses to congestion and crowding.

Therefore, we propose a unified web platform that consolidates diverse data streams into a coherent system, enabling smarter routing, real-time updates, and proactive alerts. Key functionalities include:

- **Real-Time Traffic Monitoring:** Continuous surveillance via 702 traffic cameras deployed across the city, providing live imagery for congestion detection and incident response.
- **Multi-Modal Routing Services:** Path-finding for cars, bicycles, walking, and public transit, leveraging both static road networks and dynamic traffic conditions.
- **Emergency Response:** SOS incident reporting and proximity-based alert system to improve public safety response times.

1.2 Overview of System Modules

The platform comprises four integrated modules, each serving distinct yet complementary functions:

1.2.1 Routing Module

Provides multi-modal route planning with support for:

- **Travel Modes:** Car, walking, and bicycle routing via `POST /route` endpoint.
- **Dynamic Updates:** Integrates real-time congestion data from camera analytics to adjust estimated travel times.
- **User Preferences:** Saved locations and routes stored in `user_locations` and `user_routes` tables in database, accessible through authenticated API endpoints.

1.2.2 Bus Map Module

Dedicated public transit planning interface featuring:

- **Bus Route Calculation:** `GET /api/bus/route` endpoint with configurable `max_walk` parameter (default 300 meters) for first/last mile walking distance.
- **Multi-Leg Itineraries:** Displays complete trip plans including walking segments, bus transfers, and estimated arrival times.
- **Interactive Visualization:** Goong Maps-based interface with route polylines, stop markers, and user location tracking.

1.2.3 SOS Map Module

Emergency incident reporting and visualization system:

- **Incident Reporting:** Mobile-friendly form with image upload capability via `POST /api/sos` endpoint using `FormData`.
- **Proximity Alerts:** `GET /api/sos/nearby` returns incidents within specified radius (default 500m) for location-aware notifications.
- **Status Management:** Incidents tracked with `pending/resolved` status, resolvable via `POST /api/sos/resolve` endpoint.

1.2.4 Traffic Congestion Detection Module

Computer vision-based traffic analysis:

- **Camera Network:** 702 traffic cameras with 15 minutes capture intervals, storing frames in `public/camera_frames/{camera_id}/` directory structure.
- **Metadata Tracking:** Each frame paired with JSON metadata (`camera_id`, `timestamp_iso`, hash SHA-256, `file_size`, `filename`).
- **Detection Pipeline:** Frame preprocessing, vehicle detection, lane density measurement, and congestion score calculation.
- **Real-Time Updates:** Latest frames accessible, enabling live traffic monitoring.

1.3 Motivation for Unified Data Processing Pipeline

A unified data processing pipeline addresses several critical challenges in urban transportation data management:

- **Data Fragmentation:** Multiple independent sources (cameras, transit APIs, emergency reports, geocoding services) require harmonization into coherent data models with consistent schemas and coordinate systems.
- **Scalability Requirements:** 702 cameras generating ~70000 frames/day, plus user-generated SOS reports, route requests, and location saves demand efficient storage partitioning and indexing strategies.
- **Latency Constraints:** Real-time routing and incident alerts require sub-second query response times, necessitating optimized database indexes (composite indexes on `camera_id`, `timestamp_iso` for `camera_frames`, spatial indexes for proximity queries).
- **Cross-Module Dependencies:** SOS module leverages geocoding services; bus routing integrates with general routing algorithms. Shared database layer and event-driven architecture enable seamless data flow.
- **Operational Resilience:** Centralized error handling via `tokenManager.js` for authentication failures, axios interceptors for API errors, and automated token expiration management ensure system stability despite network variability.

The pipeline architecture employs:

- **Frontend Layer:** React 17 application with Material-UI v4.12.3, Goog Maps JS SDK v1.0.9, and responsive mobile-first design.
- **Backend API:** RESTful services at `https://api.hcmus.fit` with JWT authentication, FormData handling for image uploads, and JSON response formats.
- **Data Storage:** Hybrid approach with relational database for structured data, object storage for camera frames (34 MB current), and in-memory caching for frequently accessed data.
- **State Management:** Client-side React hooks (`useState`, `useEffect`, `useRef`) for UI state, `localStorage` for authentication tokens, custom event broadcasting (`auth-change`) for cross-component synchronization.

This integrated approach enables efficient resource utilization, simplified maintenance, and extensibility for future module additions or data source integrations.

Chapter 2

Data Collection Methods and Sources

2.1 Data Sources

Bus Map Data: The data is collected by scraping directly from the `https://map.busmap.vn/hcm` web.

Traffic Camera Data: The data is collected by scraping directly from the `https://giaothong.hochiminhcity.gov.vn` web.

2.2 Data Collection Methods

2.2.1 Bus Map Data

Step-by-step implementation process:

- **Raw Capture (Interception):** A script uses playwright to launch a browser context and listen for network responses containing "api" or "route". These responses are captured into a file named `api_responses.json`.
- **Decryption:** The API responses are encrypted. The system uses the AES-CBC algorithm (via `Crypto.Cipher.AES`) to decrypt the hexadecimal payloads. The decryption key is extracted dynamically from the initial data packet.
- **Static Resource Download:** The system extracts URLs for static resources (routes, route info, stations) from the decrypted data and downloads them to a raw directory. The extractor automatically handles various compression formats, including Gzip and Zlib.
- **Tools used:** Python, Playwright (automation), PyCryptodome (decryption), Requests (HTTP calls).
- **Storage format:** The raw data is stored as JSON or binary `.xyz` files, while the final output is stored as CSV.

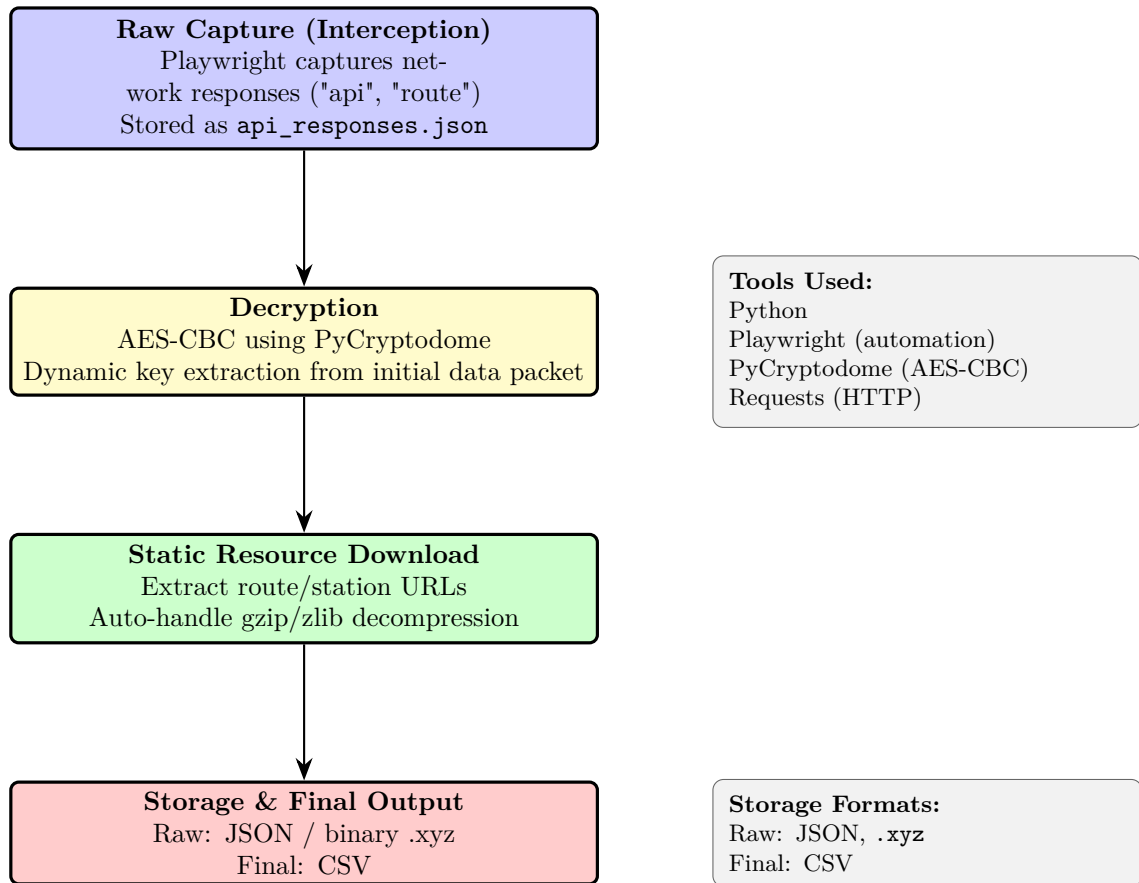


Figure 2.1: Diagram for Bus Map Data Collection

2.2.2 Traffic Camera Data

Step-by-step implementation process:

- **Command-Line Argument Parsing and Initialization:**
 - Use argparse to parse arguments: `–cameras-file` (JSON with camera list), `–interval` (cycle seconds, default 120), `–max-workers` (threads, default 6), `–output-dir` (storage root, default `./camera_frames`).
 - Load camera entries from JSON file.
 - Optionally load `camera_locations.json` for GPS and location metadata enrichment.
- **Multi-Camera Manager Setup:**
 - Instantiate `MultiCameraLoop` with camera list, interval, workers, output directory, and locations.
 - Create an `HCMCCameraCrawler` instance per camera, each with a persistent `requests.Session`, custom headers (mimicking browser), timeout (60s), and deduplication/stats tracking.
- **Signal Handling for Graceful Shutdown:** Register handlers for `SIGINT/SIGTERM` to set a stop event, allowing completion of the current cycle.
- **Continuous Looping Process:**

- Enter an infinite loop until stop event is set.
- For each cycle:
 - * Print cycle start timestamp (UTC+7).
 - * Use `ThreadPoolExecutor` (`max_workers` threads) to concurrently execute `crawl_once` for each camera.
 - * In `crawl_once`:
 - Increment attempt counter.
 - Fetch image: Construct URL with camera ID and millisecond timestamp; send GET request; validate response (status 200, image content-type, size >1000 bytes).
 - If successful: Compute SHA256 hash; check for duplication (matches previous hash); increment stats.
 - If unique: Generate timestamped filename; save JPG image and JSON metadata (camera details, timestamps, hash, size, GPS if available).
 - Cleanup: Sort images by modification time; delete all but the newest 3 (configurable) per camera folder, including companion JSON.
 - * Wait for all tasks; aggregate and print cycle stats (attempts, successes, failures, saves, duplicates).
 - * Sleep for remaining interval time (in 1-second increments to check stop event).
- **Shutdown and Final Reporting:** On stop: Complete cycle, print final per-camera stats (including uptime), and exit.

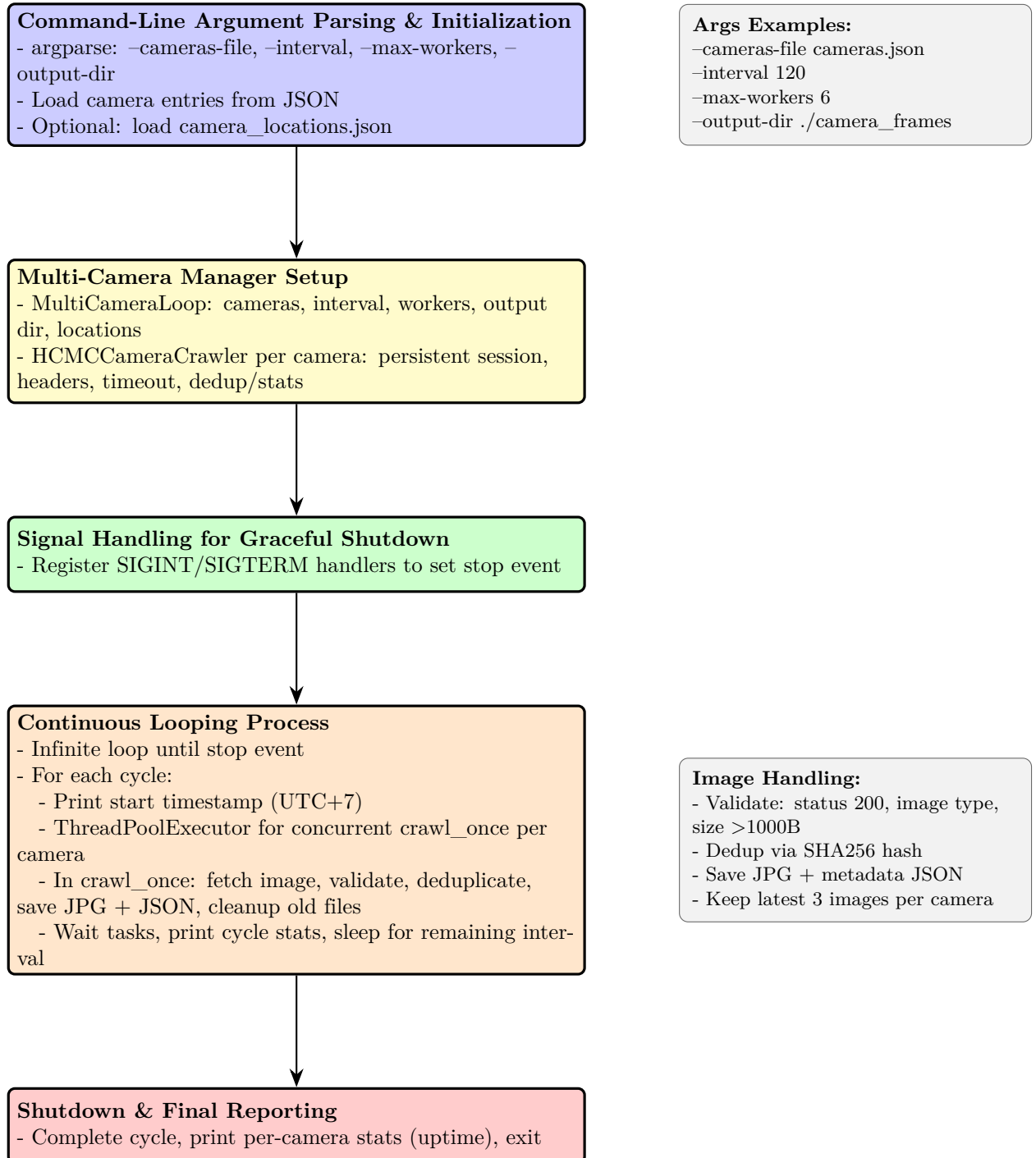


Figure 2.2: Diagram for Multi-Camera Crawling

Chapter 3

Database Management

3.1 Database Architecture

3.1.1 Relational Database (MySQL)

Primary Use Cases: Complex joins across tables (e.g., user routes with camera data for congestion awareness).

Technology Selection: MySQL: Alternative for simpler setups, wide community support, efficient for read-heavy workloads.

3.1.2 Object Storage (Google Drive Storage)

Camera Frames and Video Data:

- **Current Storage:** 35 MB for 702 latest frames in `public/camera_frames/`.
- **Production Scale:** Estimated ~3.5 GB/day (70000 frames × 50 KB).
- **Lifecycle Policies:**
 - Hot storage: Latest 7 days on local/drive storage for fast access.
 - Warm storage: 3 days on standard drive storage.

SOS Incident Images:

- User-uploaded images via `POST /api/sos FormData`.
- Store in `sos_images/` bucket with unique filenames (UUID or timestamp-based).
- Record `image_path` in `sos_reports` table for retrieval.

3.2 Schema Design

The database uses MySQL with a normalized schema consisting of four core tables: `users`, `user_locations`, `user_routes`, and `sos_alerts`. All tables use `INT AUTO_INCREMENT` primary keys and `TIMESTAMP` fields with default values for automatic tracking of record creation times.

3.2.1 Users Table

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Design Rationale:

- **id:** Auto-incrementing integer primary key for efficient joins and foreign key references.
- **username:** Unique constraint ensures no duplicate usernames; indexed automatically for login queries.
- **password:** VARCHAR(255) accommodates hashed passwords (e.g., bcrypt generates 60-character hashes, providing room for future algorithms).
- **created_at:** Tracks account registration time for analytics and compliance.

3.2.2 User Locations Table

```
CREATE TABLE IF NOT EXISTS user_locations (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    name VARCHAR(255) NOT NULL,  
    lat DOUBLE NOT NULL,  
    lng DOUBLE NOT NULL,  
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Purpose: Stores user-saved locations (home, work, frequent destinations) for quick access in routing interface.

Key Design Decisions:

- **lat/lng:** DOUBLE precision (15-16 decimal digits) provides sub-meter accuracy for geographic coordinates.
- **ON DELETE CASCADE:** Automatically removes all saved locations when user account is deleted, maintaining referential integrity.
- **name:** User-defined label for the location (e.g., “Home”, “Office”, “Gym”).

Recommended Index:

```
CREATE INDEX idx_user_locations_user_id ON user_locations(user_id);
```

Optimizes queries retrieving all saved locations for a specific user.

3.2.3 User Routes Table

```
CREATE TABLE IF NOT EXISTS user_routes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  start_name VARCHAR(255) NOT NULL,  
  start_lat DOUBLE NOT NULL,  
  start_lng DOUBLE NOT NULL,  
  end_name VARCHAR(255) NOT NULL,  
  end_lat DOUBLE NOT NULL,  
  end_lng DOUBLE NOT NULL,  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Purpose: Stores user’s route history and favorite routes for repeat navigation.

Schema Details:

- **start_name/end_name:** Human-readable labels for origin and destination (e.g., “Bến Thành Market”, “Airport”).
- **start_lat/start_lng/end_lat/end_lng:** Precise coordinates for route calculation without requiring reverse geocoding.
- **timestamp:** Records when route was saved, enabling chronological sorting in history view.

Recommended Index:

```
CREATE INDEX idx_user_routes_user_time  
ON user_routes(user_id, timestamp DESC);
```

Optimizes queries like “show last 10 routes for user X” with descending timestamp order.

3.2.4 SOS Alerts Table

```
CREATE TABLE IF NOT EXISTS sos_alerts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  lat DOUBLE NOT NULL,  
  lng DOUBLE NOT NULL,  
  description TEXT,  
  image_url VARCHAR(255),  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(20) DEFAULT 'active',  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL  
);
```

Purpose: Emergency incident reporting with optional image attachment and status tracking.

Status Workflow:

- **active:** Incident is ongoing or unresolved (default state).
- **resolved:** Incident has been addressed or is no longer relevant.

Key Design Decisions:

- **user_id:** Nullable to allow anonymous reporting; `ON DELETE SET NULL` preserves incident records even if reporting user deletes account.
- **description:** TEXT type for detailed incident descriptions (up to 65,535 characters in MySQL).
- **image_url:** Stores path to uploaded image in object storage (e.g., “`sos_images/2025/11/26/abc123`”).
- **lat/lng:** Incident location for proximity-based queries (e.g., “show all active incidents within 500m”).

Recommended Indexes:

```
CREATE INDEX idx_sos_status_time ON sos_alerts(status, timestamp DESC);
CREATE INDEX idx_sos_location ON sos_alerts(lat, lng);
```

- **idx_sos_status_time:** Optimizes queries filtering by status and sorting by time (e.g., “recent active incidents”).
- **idx_sos_location:** Enables spatial queries for proximity search (though MySQL spatial indexes with POINT type would be more efficient for large datasets).

3.3 Analytics Database

3.3.1 Primary Data Storage Schema

User Authentication Data (Table: `users`):

- **Fields:**
 - **id:** INT AUTO_INCREMENT PRIMARY KEY — unique user identifier.
 - **username:** VARCHAR(50) UNIQUE NOT NULL — login credential.
 - **password:** VARCHAR(255) NOT NULL — bcrypt-hashed password (60-character hash).
 - **created_at:** TIMESTAMP DEFAULT CURRENT_TIMESTAMP — registration time.
- **Data Volume:** Expected growth of 100-500 new users per month during pilot phase.
- **Security Constraints:** Password field never exposed in API responses; bcrypt cost factor = 10 (hashing time ~100ms).

User Location History (Table: `user_locations`):

- **Fields:**
 - `id`: INT AUTO_INCREMENT PRIMARY KEY.
 - `user_id`: INT FOREIGN KEY → `users.id` ON DELETE CASCADE.
 - `name`: VARCHAR(255) NOT NULL — user-defined label (e.g., “Home”, “Office”).
 - `lat`: DOUBLE NOT NULL — latitude coordinate (6 decimal places precision = 0.1m accuracy).
 - `lng`: DOUBLE NOT NULL — longitude coordinate.
 - `timestamp`: TIMESTAMP DEFAULT CURRENT_TIMESTAMP — save time.
- **Query Pattern:** `SELECT * FROM user_locations WHERE user_id=? ORDER BY timestamp DESC LIMIT 10` — retrieves recent locations for quick access.
- **Data Retention:** Last 10 entries per user kept; older entries archived monthly.

User Route History (Table: `user_routes`):

- **Fields:**
 - `id`: INT AUTO_INCREMENT PRIMARY KEY.
 - `user_id`: INT FOREIGN KEY → `users.id` ON DELETE CASCADE.
 - `start_name`: VARCHAR(255) NOT NULL — origin location label.
 - `start_lat`, `start_lng`: DOUBLE NOT NULL — origin coordinates.
 - `end_name`: VARCHAR(255) NOT NULL — destination label.
 - `end_lat`, `end_lng`: DOUBLE NOT NULL — destination coordinates.
 - `timestamp`: TIMESTAMP DEFAULT CURRENT_TIMESTAMP — query time.
- **Analytics Use Case:** Aggregate route queries to identify popular origin-destination pairs; informs future bus route planning.
- **Data Volume:** Average 5-10 route queries per active user per day.

SOS Alert Data (Table: `sos_alerts`):

- **Fields:**
 - `id`: INT AUTO_INCREMENT PRIMARY KEY.
 - `user_id`: INT FOREIGN KEY → `users.id` ON DELETE SET NULL — reporter (nullable for anonymous reports).
 - `lat`, `lng`: DOUBLE NOT NULL — incident location.
 - `description`: TEXT — user-provided details (max 65,535 characters).
 - `image_url`: VARCHAR(255) — path to uploaded photo in object storage (e.g., `sos_images/2025/11/27/abc123.jpg`).
 - `timestamp`: TIMESTAMP DEFAULT CURRENT_TIMESTAMP — report time.
 - `status`: VARCHAR(20) DEFAULT 'active' — lifecycle state: **active** (visible on map) or **resolved** (archived).

- **Spatial Indexing:** Composite index on (lat, lng, status) for fast proximity queries (WHERE status='active' AND lat BETWEEN ... AND lng BETWEEN ...).
- **Data Cleanup:** resolved alerts older than 30 days archived to cold storage; images moved to S3 Glacier.

3.3.2 Analytics Metrics and Aggregated Data

Camera Frame Metadata (Table: camera_frames):

- **Fields:**
 - id: INT AUTO_INCREMENT PRIMARY KEY.
 - camera_id: VARCHAR(50) NOT NULL — unique camera identifier from HCMC traffic system.
 - filename: VARCHAR(255) NOT NULL — stored image filename.
 - timestamp_iso: VARCHAR(50) NOT NULL — ISO 8601 capture time (e.g., 2025-11-27T14:30:00Z).
 - file_size: INT — image file size in bytes.
 - hash_prefix: VARCHAR(20) — first 8 characters of SHA-256 hash for deduplication.
- **Query Optimization:** Index on (camera_id, timestamp_iso DESC) enables fast retrieval of latest frames per camera.
- **Data Volume:** 1,500 cameras × 288 frames/day (5-minute intervals) = 432,000 records/day.

Congestion Score Logs (Table: congestion_logs):

- **Fields:**
 - id: INT AUTO_INCREMENT PRIMARY KEY.
 - camera_id: VARCHAR(50) NOT NULL.
 - timestamp: TIMESTAMP NOT NULL — analysis time.
 - congestion_score: FLOAT(3,2) — normalized score 0.00-1.00 (e.g., 0.68 = moderate-heavy congestion).
 - vehicle_count: INT — detected vehicles in frame (optional supplementary metric).
- **Derived Categories:** Traffic intensity classification applied at query time:
 - 0.00-0.30: Free flow (green marker).
 - 0.31-0.60: Moderate (yellow marker).
 - 0.61-0.80: Heavy (orange marker).
 - 0.81-1.00: Severe (red marker).
- **Aggregation Queries:**
 - Hourly average: SELECT AVG(congestion_score) FROM congestion_logs WHERE camera_id=? AND timestamp >= NOW() - INTERVAL 1 HOUR.

- Peak congestion time: `SELECT HOUR(timestamp), AVG(congestion_score) GROUP BY HOUR(timestamp) ORDER BY AVG DESC LIMIT 1.`
- **Data Retention:** Raw logs kept 7 days; hourly aggregates stored 90 days; daily averages archived indefinitely.

Bus ETA Performance Data (Table: `bus_eta` — Optional):

- **Fields:**
 - `id`: INT AUTO_INCREMENT PRIMARY KEY.
 - `route_id`: VARCHAR(20) NOT NULL — bus route identifier.
 - `stop_id`: VARCHAR(20) NOT NULL — bus stop ID.
 - `estimated_arrival`: TIMESTAMP NOT NULL — predicted arrival time.
 - `actual_arrival`: TIMESTAMP — observed arrival (NULL if not yet arrived).
 - `prediction_error`: INT — difference in seconds (calculated post-arrival).
- **Analytics Metrics:**
 - Mean Absolute Error (MAE): `AVG(ABS(prediction_error))` per route.
 - Accuracy rate: Percentage of predictions within ± 3 minutes of actual arrival.
- **Use Case:** Model retraining input — identify routes with high prediction errors for algorithm tuning.

3.3.3 Cache Layer for Performance Optimization

Redis Key-Value Data Structures:

- **Camera Frame Cache:**
 - Key pattern: `cache:camera:{camera_id}:latest`
 - Value: JSON string `{"url": "/camera_frames/56de42f6.../...", "timestamp": "2025-11-27T14:30:00Z", "congestion_score": 0.68}`
 - TTL: 300 seconds (5 minutes) — matches frame update frequency.
 - Query avoidance: Eliminates 95% of database reads for latest frame requests.
- **Geocoding Result Cache:**
 - Key pattern: `cache:geocode:{address_hash}` (SHA-256 hash of normalized address).
 - Value: JSON `{"lat": 10.7724, "lon": 106.6980, "formatted_address": "..."}`
 - TTL: 86400 seconds (24 hours) — addresses rarely change.
 - Cost savings: Reduces TomTom API calls by 70% (geocoding quota = 2,500 requests/day).
- **Route Computation Cache:**
 - Key pattern: `cache:route:{start_lat}:{start_lon}:{end_lat}:{end_lon}:{mode}`

- Value: Serialized route data (coordinates array, distance, duration).
- TTL: 3600 seconds (1 hour) — balances freshness with cache efficiency.
- Grid snapping: Coordinates rounded to 4 decimal places (11m precision) to increase cache hit rate.
- **SOS Alert Proximity Cache:**
 - Key pattern: `cache:sos:active:{grid_cell_id}` (geohash grid cells, 500m × 500m).
 - Value: JSON array of active alerts in cell `[{id, lat, lng, severity, timestamp}, ...]`
 - TTL: 60 seconds — near-real-time updates for user safety.

Cache Invalidation Strategies:

- **Time-Based Expiration:** Automatic TTL eviction — no manual intervention required for most keys.
- **Event-Driven Invalidation:**
 - New camera frame ingestion → `DEL cache:camera:{id}:latest` + `SET` new data.
 - SOS alert status change (`active` → `resolved`) → `DEL` all affected geohash cell keys.
- **Write-Through Pattern:** For critical data (SOS alerts), write to MySQL first, then update Redis cache — ensures durability before cache update.

Cache Performance Metrics:

- **Hit Rate Target:** $\geq 85\%$ for camera frames, $\geq 90\%$ for geocoding.
- **Latency Improvement:** Average query time reduced from 15-50ms (MySQL) to $< 1\text{ms}$ (Redis).
- **Memory Footprint:** Estimated 2-4 GB Redis memory for 10,000 cached routes + 1,500 camera states + 50,000 geocoding results.

3.3.4 Data Export and Reporting Formats

Daily Aggregate Reports (CSV):

- **Congestion Summary:** `camera_id, date, avg_congestion_score, peak_hour, peak_score, free_flow_hours`
- **Route Query Stats:** `date, total_queries, unique_users, top_origin, top_destination, avg_distance_km`
- **SOS Incident Log:** `date, total_reports, resolved_count, avg_resolution_time_min, severity_breakdown`

JSON API Response Schemas:

- **User Route History:** GET /api/user/routes returns [{id, start_name, start_lat, start_lng, end_name, end_lat, end_lng, timestamp}, ...] (last 10 entries).
- **Active SOS Alerts:** GET /api/sos returns [{id, user_id, username, lat, lng, description, image_url, timestamp, status}, ...] (filtered by status='active').
- **Camera Statistics:** GET /api/stats returns {total_cameras: 1500, cameras_with_images: 1420, total_images: 432000}.

Chapter 4

Data Cleaning and Preprocessing Workflow

4.1 Bus Map Data Cleaning and Preprocessing

- **Remove Noise and Duplicates**

Station Deduplication: The `stops_export.py` script removes duplicate stations. It prioritizes a unique `stopId`; if that is absent, it deduplicates based on a unique combination of coordinates (rounded to 6 decimal places) and station names.

- **Remove or Handle Missing Values**

- Distance Calculation: In the `trips_export.py` script, if the API does not provide a `distanceHint` between stops, the system calculates the distance manually using the Haversine formula based on the latitude and longitude of the sequential stops.
- Time Normalization: Route operation times (e.g., "05:30 - 20:30") are split into distinct `activeStartTime` and `activeStopTime` columns. If data is missing, these default to empty strings.

- **Standardization / Transformation**

- Data Type Conversion:
 - * Coordinates (Lat/Lng) are explicitly converted to floats.
 - * Ticket fees (`NormalTicket`) are parsed from text strings into numeric values.
- Trip Time Formatting: The `TimeOfTrip` field is normalized. The script resolves inconsistencies between outbound and inbound timings by creating a standard range format (e.g., "50-55") or a single integer string.

4.2 Traffic Camera Data Cleaning and Preprocessing

- **Remove Duplicates**

- Use SHA-256 hash to identify identical frames.
- Skip saving duplicate frames, update metadata with latest timestamp instead.
- Useful when camera returns same static image due to connection issues.

- **Standardization / Transformation**

- Text Data:
 - * Ensure all text fields (camera names, addresses, descriptions) use UTF-8 encoding.
 - * Handle Vietnamese diacritics correctly (e.g., “Mai Chí Thọ - Trần Nãi 1”).
 - * Normalize whitespace: strip leading/trailing spaces, collapse multiple spaces to single.
- JSON Structure:
 - * Validate JSON schema for camera metadata: require camera_id, timestamp_iso, hash fields.
 - * Reject malformed JSON with parse errors, log for investigation.
 - * Apply default values for optional fields (e.g., display_name defaults to camera_name if missing).

Chapter 5

Exploratory Data Analysis

5.1 Descriptive Statistics

5.1.1 Geospatial Distribution

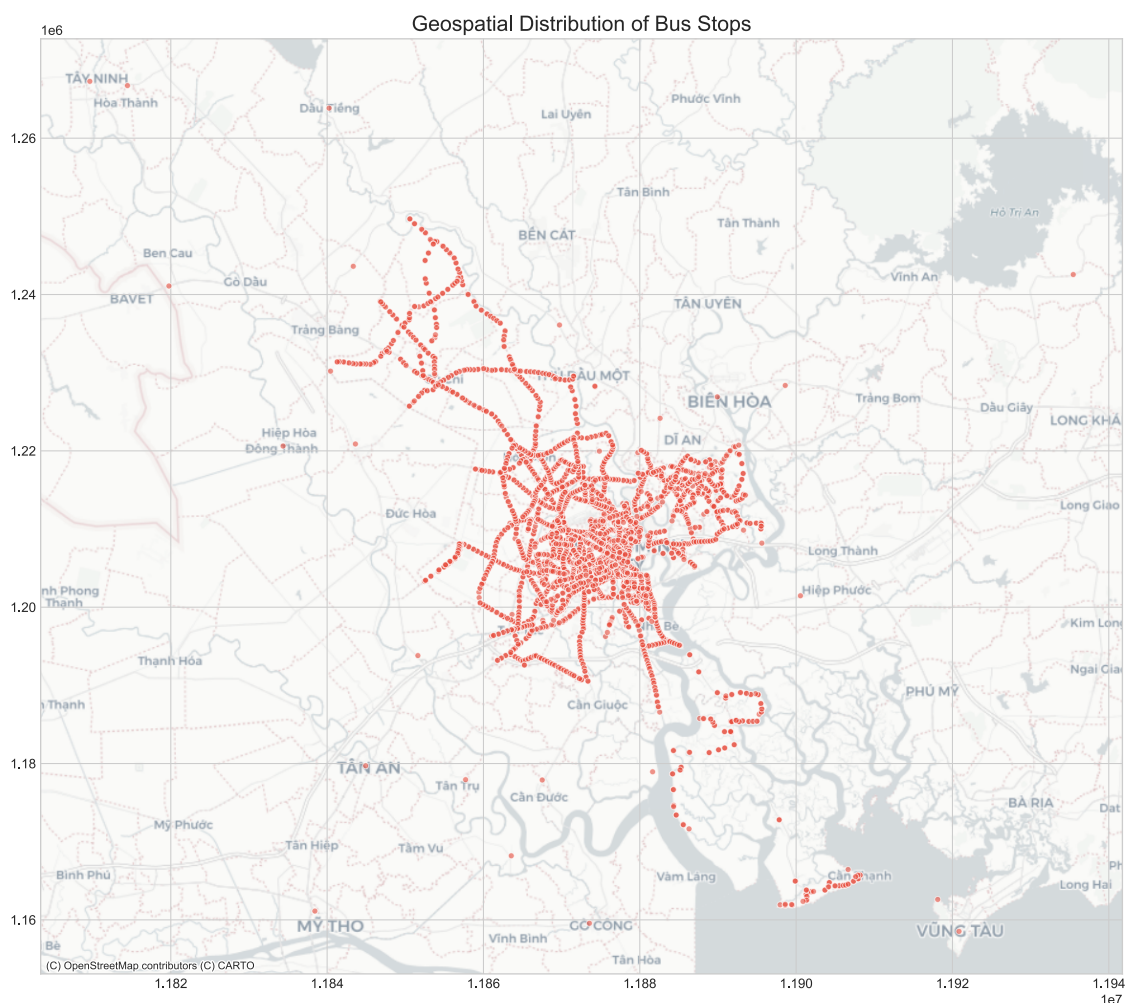


Figure 5.1: Geographical Distribution of Bus Stops in Ho Chi Minh City

This report provides a descriptive analysis of the Ho Chi Minh City bus network based on the provided `stops.csv` dataset and geospatial visualizations. The network consists of **4,343 unique bus stops**, serving as the primary public transport nodes for the city.

Network Scale & Geographic Coverage

The bus network covers a wide geographic area, extending from the dense city center to suburban and rural districts.

- **Total Stops:** 4,343
- **Geographic Center:** Latitude 10.80°N, Longitude 106.66°E (approx. Tan Binh/-Phu Nhuan District).
- **North-South Extent:** ~ 106 km (From 10.35°N to 11.31°N).
- **East-West Extent:** ~ 123 km (From 106.09°E to 107.22°E).

Location Type Analysis

By analyzing common keywords in the stop names, we identified the primary types of infrastructure served by the bus network. The data indicates a strong focus on educational institutions and markets.

Table 5.1: Distribution of Stops by Location Type

Location Type	Common Keywords	Count
Education	<i>Trường, Đại học, Cao đẳng</i>	482
Spiritual/Community	<i>Chùa, Nhà thờ, Đình</i>	368
Workplaces	<i>Công ty, Khu công nghiệp</i>	300
Commerce	<i>Chợ, Siêu thị</i>	256
Healthcare	<i>Bệnh viện, Trạm y tế</i>	186
Transport Hubs	<i>Bến xe, Ga</i>	119

Core Density: Approximately 50% of all bus stops are concentrated within a high-density urban core defined by the interquartile range:

- **Latitude Range:** 10.75°N – 10.85°N
- **Longitude Range:** 106.61°E – 106.71°E

This area corresponds to the central districts (D1, D3, D5, D10, Phu Nhuan, and Binh Thanh).

Outliers and Remote Connectivity: The analysis identified **344 geographical outliers**. These stops represent the network's reach into remote or rural areas, such as:

- **North:** Cu Chi and Tay Ninh border (e.g., stops like *Tỉnh lộ 6*).
- **South:** Can Gio biosphere reserve (sparse connectivity).
- **West:** Binh Chanh and Long An border.

Visual Structure

Based on the generated map above, the network exhibits a **radial structure**:

1. A dense central cluster representing the urban core.
2. Linear extensions (“tentacles”) following major highways (e.g., Highway 22, Highway 1A) connecting the center to satellite towns.

5.1.2 Numerical Variables

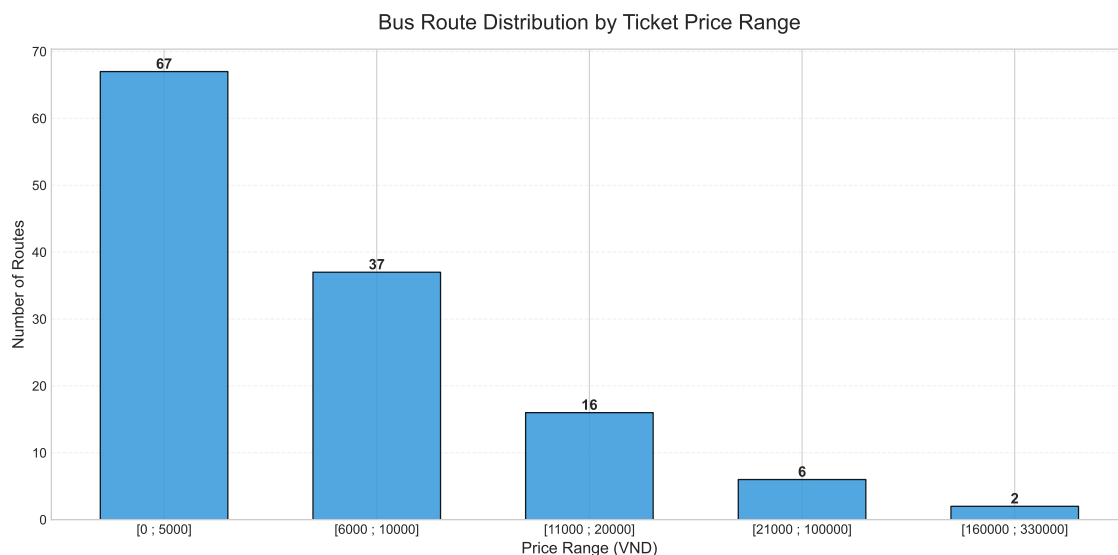


Figure 5.2: Distribution of Ticket Fees for Bus Routes

Ticket Fees Analysis The majority of bus routes are affordable city commuters, but there are significant outliers representing long-distance or tourism services.

- Average Fee: 12,500 VND
- Most Common Fee (Median): 6,000 VND
- Price Range: 5,000 VND (min) to 330,000 VND (max)

Most Expensive Routes:

- City Tour (Tuyến xe du lịch vòng khu vực trung tâm): 330,000 VND - This is a sightseeing hop-on hop-off service, not a standard commute.
- Airport to Vung Tau (Sân bay TSN – Bến xe Vũng Tàu): 160,000 VND - A premium intercity link.
- Can Gio - Vung Tau Ferry Link: 70,000 VND.

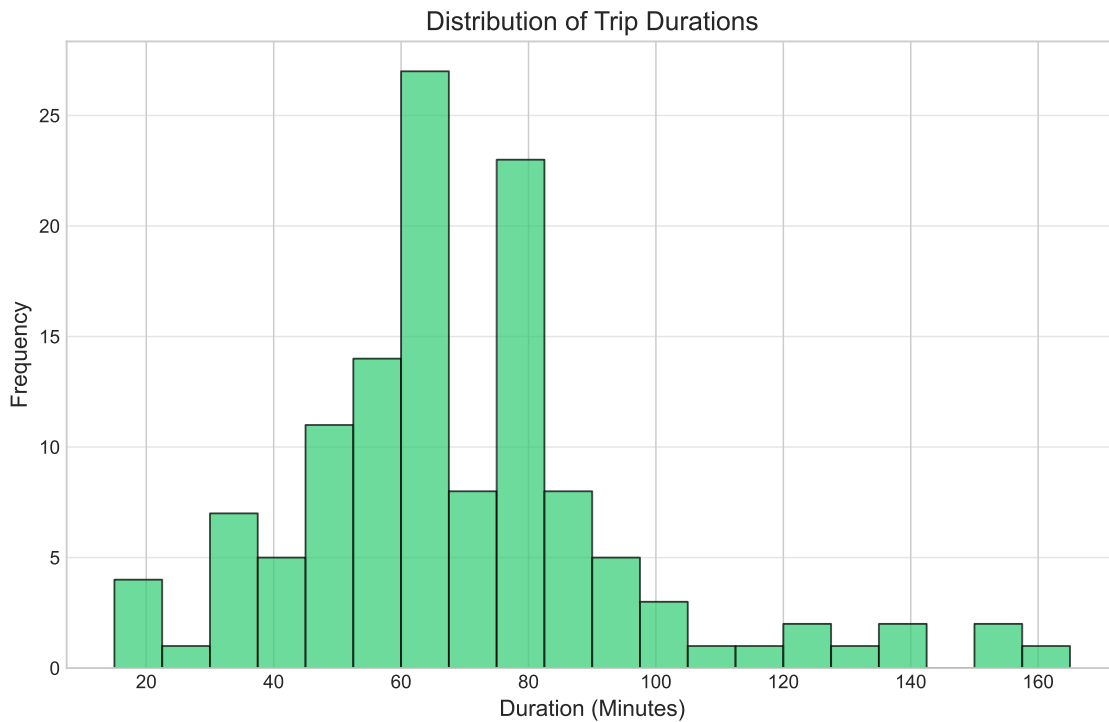


Figure 5.3: Distribution of Trip Durations for Bus Routes

Trip Duration Analysis The `busStopSpacing(time of trip)` column reveals how long passengers typically spend on a bus for a full route.

- Average Duration: 68 minutes
- Standard Commute: Most trips last between 55 and 80 minutes.
- Shortest Route: 15 minutes (likely a short shuttle or feeder service).
- Longest Route: 165 minutes (2.75 hours).

Based on the analysis, we build an algorithm to estimate the travel time that nearly matches the actual trip duration provided in the dataset.

- Implements a modified **A*** search algorithm that considers:
 - Distance between stops.
 - Transfer penalties between routes.
 - Starting penalty for initial waiting time.
 - A **transfer penalty** is added when switching between routes.
- The cost function is defined as:

$$\text{cost} = \text{currentcost} + \frac{\text{edgeDist}}{v} + \text{transferPen} \cdot p + c \cdot \text{fare}$$

- **currentcost**: accumulated cost to reach the current node.
- **edgeDist**: distance (in meters) of the current bus segment.

- **transferPen**: transfer penalty, equal to the average bus spacing / waiting time; set `transfer_pen = 0` if there's no transfer between routes.
- v (m/s): average bus speed, assumed to be **7 m/s**.
- p : constant coefficient that amplifies the importance of bus transfer penalties.
- c : constant coefficient that amplifies the importance of bus fare.
- **fare**: bus fare cost; fare = 0 if there's no route transfer.

$$\text{WorstCaseDuration} = \text{TotalCost} - c \cdot \text{TotalFares} - (p - 1) \cdot \text{TotalWait} + \text{walkDistance} \cdot \text{walkSpeed}$$

$$\text{BestCaseDuration} = \text{WorstCaseDuration} - \text{TotalWait}$$

The algorithm can compare multiple potential starting stops.

5.2 Analysis of Relationships

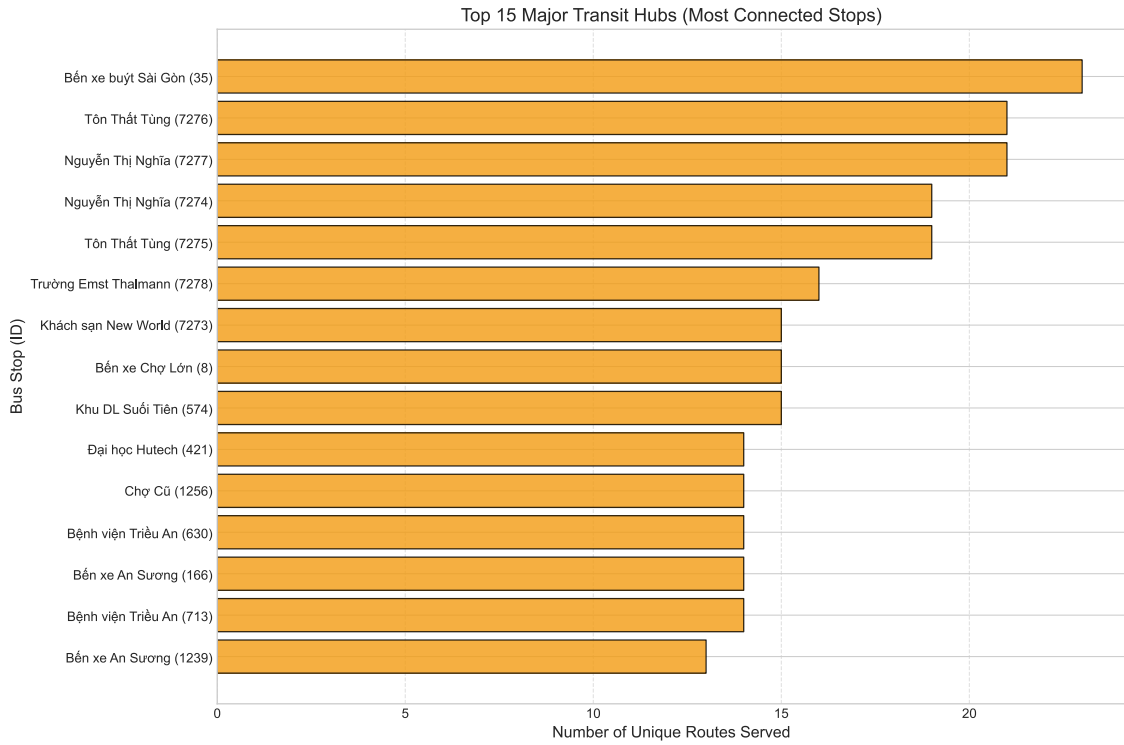


Figure 5.4: Proximity of Bus Stops to Major Transit Hubs in Ho Chi Minh City

Based on the `trips.csv` dataset, we analyzed the physical connections (edges) that define the bus network structure.

Network Scale

- **Nodes (Stops)**: 4,343 unique locations.
- **Edges (Links)**: 10,459 connections between stops.

- **Routes:** 129 distinct bus lines.

Distance Dynamics (Edge Weights)

- **Average Hop:** The average distance between two consecutive stops is **~577 meters**.
- **Short Hops:** 25% of stops are less than **269 meters** apart. This indicates a highly dense network in the city center where stops are frequently placed to facilitate easy access.
- **Long Hops:** The data contains extreme outliers (up to 63 km), representing express routes or intercity connections.

Route Complexity

- **Average Route Length:** ~46.8 km (Total round trip).
- **Variation:** Routes exhibit significant variance, ranging from short shuttle services (5.5 km) to long-haul provincial connections (146 km).

Hub Identification Using stop connectivity data, we identified key transit hubs in the network:

The Super-Hub (Stop ID 35)

- **Name:** Bến xe buýt Sài Gòn (Ben Thanh Bus Station).
- **Connectivity:** Connected to **23 unique routes**.
- **Role:** This node serves as the absolute center of the network. Routing algorithms should prioritize this node as a primary transfer point.

The Central-Cluster (Stops 7276, 7277, 7274)

- **Names:** Tôn Thất Tùng, Nguyễn Thị Nghĩa.
- **Connectivity:** Connected to **19–21 routes**.
- **Insight:** Geographically adjacent to Ben Thanh Market, these stops act as overflow terminals, effectively forming a larger “Central Transit Zone.”

Key Regional Gateways

- **Stop 8 (Bến xe Chợ Lớn):** 15 routes – The primary hub for the West/Chinatown area.
- **Stop 574 (Khu DL Suối Tiên):** 15 routes – The gateway to the East and University Village.
- **Stop 166 (Bến xe An Sương):** 14 routes – The gateway to the Northwest.

Conclusion The combination of connectivity data and hub identification reveals a **Hub-and-Spoke** network architecture:

1. **Core:** A massive concentration of connections at the city center (Stop ID 35 and surroundings).
2. **Spokes:** Long routes (avg 46 km) radiate outwards to regional terminals (Cho Lon, An Suong, Suoi Tien).
3. **Implication:** Efficient navigation across the city usually requires transferring through one of the top 15 identified hubs.

Chapter 6

EDA For Unstructured Data

6.1 Traffic Camera Image Analysis

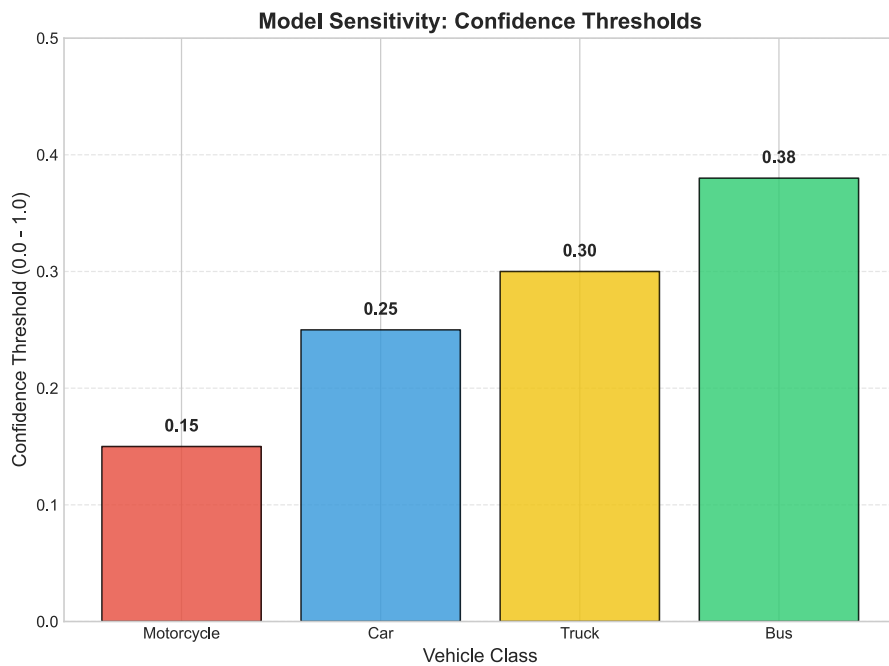


Figure 6.1: Model Sensitivity Analysis

Table 6.1: Optimized Detection Thresholds by Vehicle Class

Class	Threshold	Sensitivity	Objective
Motorcycle	0.15	High	Maximize detection in dense/occluded traffic
Car	0.25	Medium	Standard detection baseline
Truck	0.30	Low	Reduce confusion with large passenger vehicles
Bus	0.38	Very Low	Ensure high precision for routing data integrity

Analysis: The motorcycle threshold is set aggressively low (0.15) to mitigate occlusion issues common in HCMC’s mixed traffic flow. Conversely, the bus threshold is stringent (0.38) to prevent false positives from corrupting the public transport routing database.

6.2 Traffic Congestion Data Pipeline

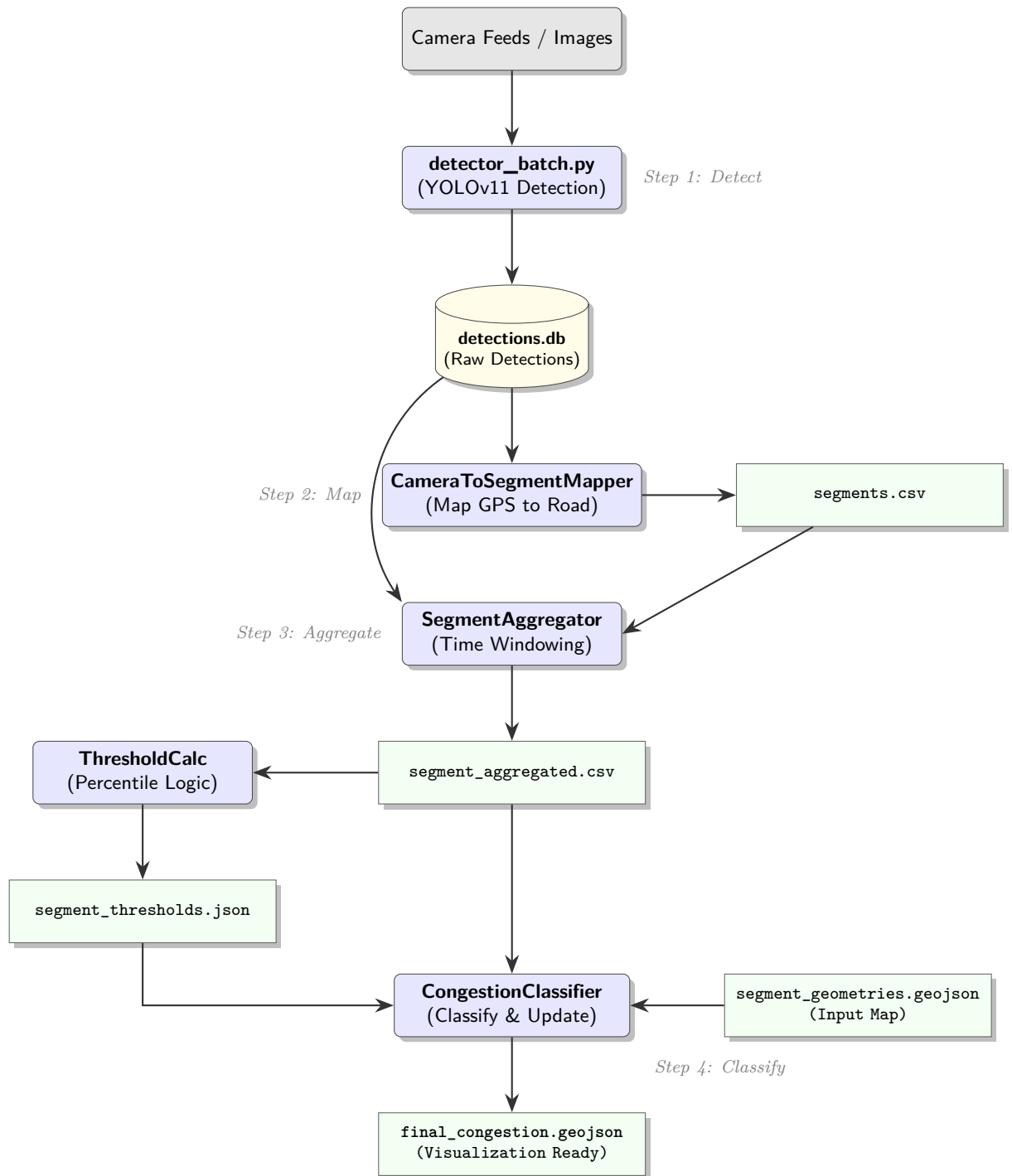


Figure 6.2: Traffic Congestion Data Pipeline Architecture